

```
In [ ]: metaproject_name = 'HYPNOMED_META'
        subproject_name = 'hypnomed_pheno'
```

```
In [ ]: import os
import pandas as pd
import numpy as np

PROJDIR = os.path.expanduser(f"~/projects/{metaproject_name}")
DATADIR = os.path.join(PROJDIR, f'DATA/{subproject_name}')
CODEDIR = os.path.join(PROJDIR, f'{subproject_name}')
```

```
In [ ]: file_name = os.path.join(DATADIR, 'Pheno_Likert_scales_fMRI_Romy.csv')
print(file_name)
Likert_df = pd.read_csv(file_name, header = 0)
Likert_df
```

/Users/rb666/projects/HYPNOMED_META/DATA/hypnomed_pheno/Pheno_Likert_scales_fMRI_Romy.csv

```
Out[ ]:
```

	Sujet	Sub_id_fMRI	Neucose_ID	Bloc	state	Version	age	genre	allcov_e
0	1	sub-25	KA001	1	C	B	034Y	F	
1	5	sub-40	AV005	1	C	A	022Y	F	
2	7	sub-34	AV007	1	C	A	026Y	M	
3	14	sub-33	JC014	1	C	B	022Y	F	
4	21	sub-08	GM021	1	C	B	063Y	M	
...	...	...	...	...	...	...	...	...	
115	94	sub-18	LD094	2	H	B	045Y	F	
116	96	sub-39	BR096	3	H	A	032Y	M	
117	100	sub-11	GP100	3	H	A	061Y	M	
118	101	sub-31	DM101	2	H	B	047Y	F	
119	102	sub-24	AB102	2	H	B	022Y	M	

120 rows × 36 columns

Modify the dataset so that it matches the nomenclature of Bigdataset.txt + Compute the Likert values

```
In [ ]: Likert_df.keys()

# Creating new columns
Likert_categories = ['difficulty', 'emotion', 'absorp', 'meta', 'realness'] #

Likert_df['difficulty'] = Likert_df['Stateq_Answ1'].astype(int) + Likert_
Likert_df['emotion'] = Likert_df['Stateq_Answ4'].astype(int) + Likert_df[
Likert_df['absorp'] = Likert_df['Stateq_Answ11'].astype(int) + Likert_df[
Likert_df['meta'] = Likert_df['Stateq_Answ16'].astype(int) + Likert_df['S
```

```

Likert_df['realness'] = Likert_df['Stateq_Answ6'].astype(int) + Likert_df

Likert_mapping = {
    "X01_following_instructions": "Stateq_Answ1",
    "X02_difficulty": "Stateq_Answ2",
    "X03_distractedness": "Stateq_Answ3",
    "X04_emotivity": "Stateq_Answ4",
    "X05_memories": "Stateq_Answ5",
    "X012_emotional_implication_in_sensations": "Stateq_Answ12",
    "X011_intensity_of_sensation": "Stateq_Answ11",
    "X013_absorption_in_foods": "Stateq_Answ13",
    "X014_absorption_in_sensation": "Stateq_Answ14",
    "X016_observing_thoughts": "Stateq_Answ16",
    "X017_thoughts_as_interpretations_of_reality": "Stateq_Answ17",
    "X06_appetite": "Stateq_Answ6",
    "X07_mouthwatering": "Stateq_Answ7",
    "X08_realness_of_pictures": "Stateq_Answ8",
    "X09_appetite_increase": "Stateq_Answ9",
    "X010_realness_of_sensation": "Stateq_Answ10",
    "X015_realness_of_sensations": "Stateq_Answ15",
}

# Copy the dataframe
df = Likert_df.copy()

# Reverse the column mapping
reverse_mapping = {v: k for k, v in Likert_mapping.items()}

# Define a list of other columns that you want to keep
other_columns = ["Sujet", "Bloc", "state", "Version", "genre", "age", "all

# Create a new dataframe with the columns renamed according to the revers
df_renamed = df.rename(columns=reverse_mapping)

# Create a list of columns in the desired order
ordered_columns = other_columns + [col for col in df_renamed.columns if c

# Reorder the columns in the dataframe
new_df = df_renamed[ordered_columns]
Likert_questions = list(key for key in new_df.keys() if key.startswith('X
print(Likert_questions)

```

```

['X01_following_instructions', 'X02_difficulty', 'X03_distractedness', 'X0
4_emotivity', 'X05_memories', 'X06_appetite', 'X07_mouthwatering', 'X08_re
alness_of_pictures', 'X09_appetite_increase', 'X010_realness_of_sensatio
n', 'X011_intensity_of_sensation', 'X012_emotional_implication_in_sensatio
ns', 'X013_absorption_in_foods', 'X014_absorption_in_sensation', 'X015_rea
lness_of_sensations', 'X016_observing_thoughts', 'X017_thoughts_as_interpr
etations_of_reality']

```

Delta compute

Compute the delta between states and Control condition. So :

- $\text{Value_delta} = \text{Value_Meditation} - \text{Value_Control}$

```
In [ ]: # Creating data.ctrl dataframe : Only data for Control condition
data_ctrl = new_df.loc[new_df['Bloc'] == 1].drop_duplicates()
# data_ctrl = data_ctrl[['Sujet', 'Version', 'state', 'Bloc', 'age', 'genre', 'difficulty', 'emotion', 'absorp', 'meta', 'realness']]
data_ctrl = data_ctrl[['Sujet', 'difficulty', 'emotion', 'absorp', 'meta', 'realness']]
data_ctrl = data_ctrl.rename(columns={
    'difficulty': 'Ctrl_difficulty',
    'emotion': 'Ctrl_emotion',
    'absorp': 'Ctrl_absorp',
    'meta': 'Ctrl_meta',
    'realness': 'Ctrl_realness'
})
data_ctrl.head()
```

```
Out [ ]:
```

	Sujet	Ctrl_difficulty	Ctrl_emotion	Ctrl_absorp	Ctrl_meta	Ctrl_realness
0	1	17	5	6	2	23
1	5	21	3	3	3	8
2	7	20	22	17	17	33
3	14	18	8	13	13	32
4	21	12	17	13	13	24

```
In [ ]: # Creating data.delta dataframe : Only data for non control condition
data_delta = new_df.loc[new_df['Bloc'] != 1]
data_delta = pd.merge(data_delta, data_ctrl, on='Sujet', how='left')
data_delta['difficulty_delta'] = data_delta['difficulty'] - data_delta['Ctrl_difficulty']
data_delta['emotion_delta'] = data_delta['emotion'] - data_delta['Ctrl_emotion']
data_delta['absorp_delta'] = data_delta['absorp'] - data_delta['Ctrl_absorp']
data_delta['meta_delta'] = data_delta['meta'] - data_delta['Ctrl_meta']
data_delta['realness_delta'] = data_delta['realness'] - data_delta['Ctrl_realness']
data_delta = data_delta[['Sujet', 'Bloc', 'age', 'genre', 'Version', 'state', 'difficulty_delta', 'emotion_delta', 'absorp_delta', 'meta_delta', 'realness_delta']]

# Removing donne.ctrl
del data_ctrl
```

STATS

Functions definition

- Boxplot : provides a visual representation of the distribution of the variable of interest across different blocks ('Bloc') and states ('state')
- Kernel Density Estimate plot : provides a smoothed representation of the distribution of the variable of interest across different states. The use of kernel smoothing to plot values allow for smoother distributions by smoothing out the noise. The peaks of a density plot help display where values are concentrated over the interval.
- Linear regression with the ordinary least squares (OLS) method : The dependent variable is the variable of interest, and the independent variables are the interaction between the 'state' and 'Bloc'.

```
In [ ]: import statsmodels.api as sm
import statsmodels.formula.api as smf
import seaborn as sns
import matplotlib.pyplot as plt
from statsmodels.sandbox.stats.multicomp import MultiComparison
from statsmodels.stats.multitest import multipletests

def analysis_al_python(var_interest, datas):
    datas['col_interest'] = datas[var_interest]

    # boxplot
    sns.boxplot(x='Bloc', y='col_interest', hue='state', data=datas) #data
    plt.title(var_interest)
    plt.show()

    # density plot : Plot univariate or bivariate distributions using kernel
    sns.kdeplot(data=datas, x='col_interest', hue='state')
    plt.title(var_interest)
    plt.show()

    # OLS regression
    model = smf.ols('col_interest ~ state*Bloc', data=datas).fit()
    print(model.summary())

    # ANOVA results
    anova_table = sm.stats.anova_lm(model, typ=2)
    print(anova_table)

    # plot model
    # Here we will plot the predictions from our model
    predictions = model.predict(datas[['state', 'Bloc']])
    sns.lineplot(data=datas, x='Bloc', y=predictions, hue=datas['state'])
    plt.title('Predicted Values')
    plt.show()

    # Post-hoc testing with multiple comparisons
    mc = MultiComparison(datas['col_interest'], datas['state'])
    result = mc.tukeyhsd()

    # Adjust the p-values from the Tukey HSD test using the FDR method
    reject, pvals_corrected, _, _ = multipletests(result.pvalues, method='fdr')
```

```

print('FDR corrected p-values: ', pvals_corrected)

# Defining contrasts
contrasts = {
    "Abso_3 VS Abso_2": np.array([-1, 0, 1, 0]),
    "Meta_3 VS Meta_2": np.array([0, -1, 0, 1]),
    "Abso_2 VS Meta_2": np.array([1, -1, 0, 0]),
    "Abso_3 VS Meta_3": np.array([0, 0, -1, 1])
}

# Store everything in a dictionary
output = {
    'anova_table': anova_table,
    'predictions_plot': plt,
    'emerg_test': pvals_corrected,
    'post_hoc': contrasts # This would usually include the contrast r
}

return output

```

Run analysis

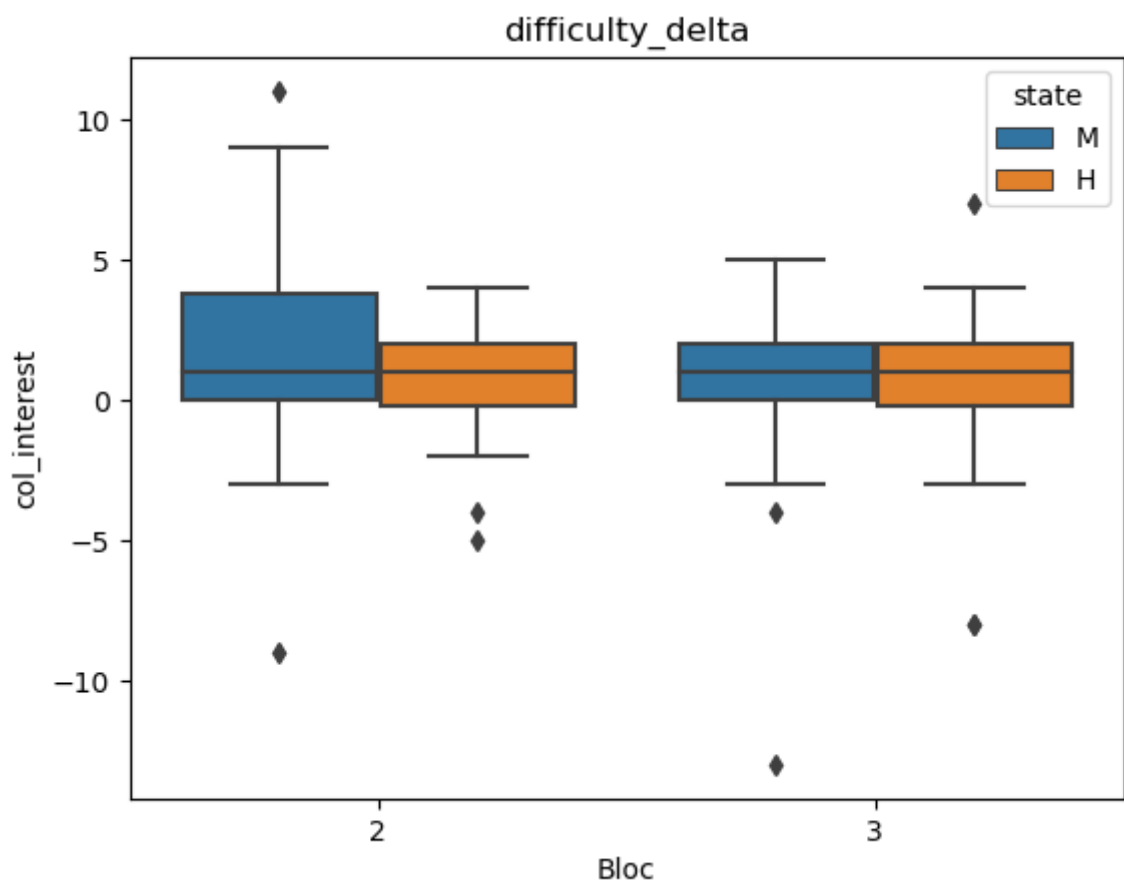
```

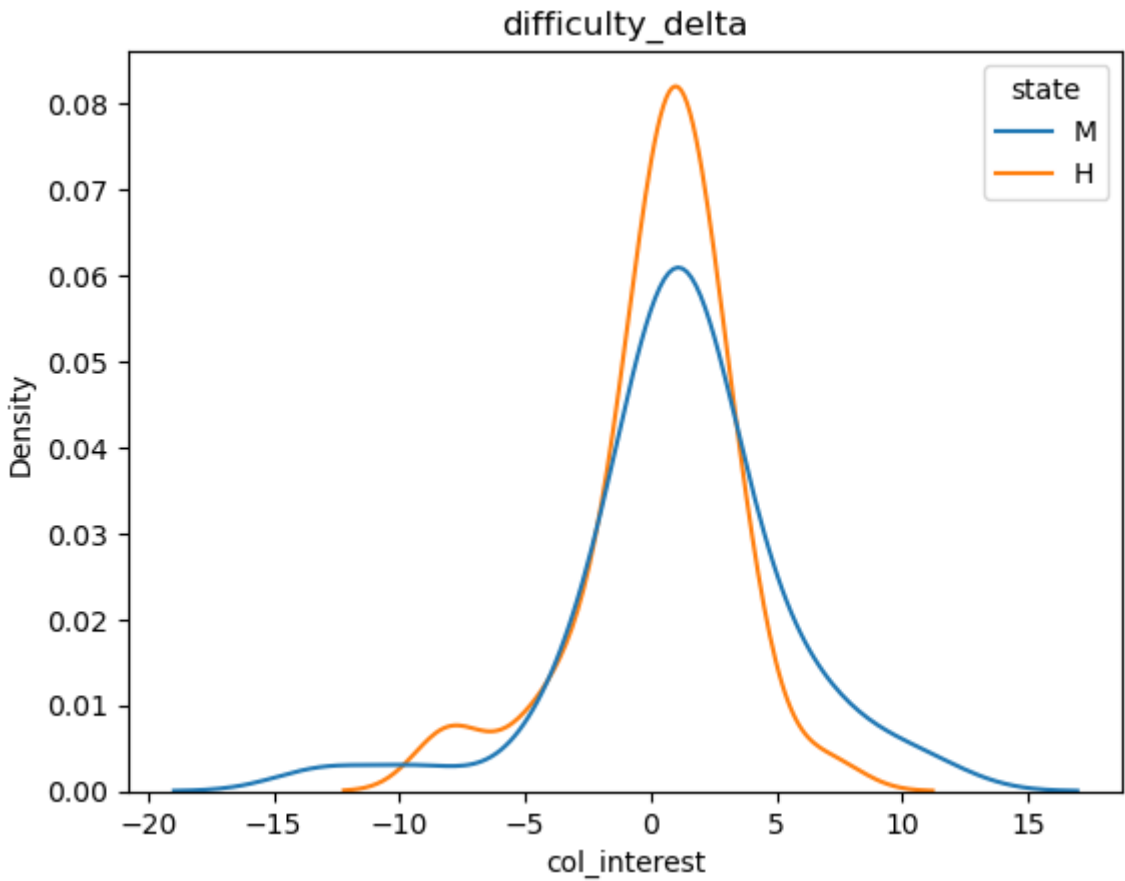
In [ ]: # Iterate through the output list and print results for each variable
from IPython.display import display

var_to_analys = ["difficulty_delta", "emotion_delta", "absorp_delta", "me

# Apply analysis_al function to each variable
output = [analysis_al_python(var, data_delta) for var in var_to_analys]

```





OLS Regression Results

```

=====
=====
Dep. Variable:          col_interest    R-squared:
0.040
Model:                  OLS             Adj. R-squared:
0.002
Method:                 Least Squares    F-statistic:
1.049
Date:                   Tue, 06 Jun 2023  Prob (F-statistic):
0.376
Time:                   15:32:33          Log-Likelihood:          -21
3.71
No. Observations:      80                AIC:                      4
35.4
Df Residuals:          76                BIC:                      4
45.0
Df Model:               3
Covariance Type:       nonrobust
=====
=====

```

	coef	std err	t	P> t	[0.025
Intercept	0.8500	2.894	0.294	0.770	-4.914
state[T.M]	4.5000	4.093	1.099	0.275	-3.652
Bloc	-0.2000	1.135	-0.176	0.861	-2.461
state[T.M]:Bloc	-1.5000	1.605	-0.934	0.353	-4.697

```

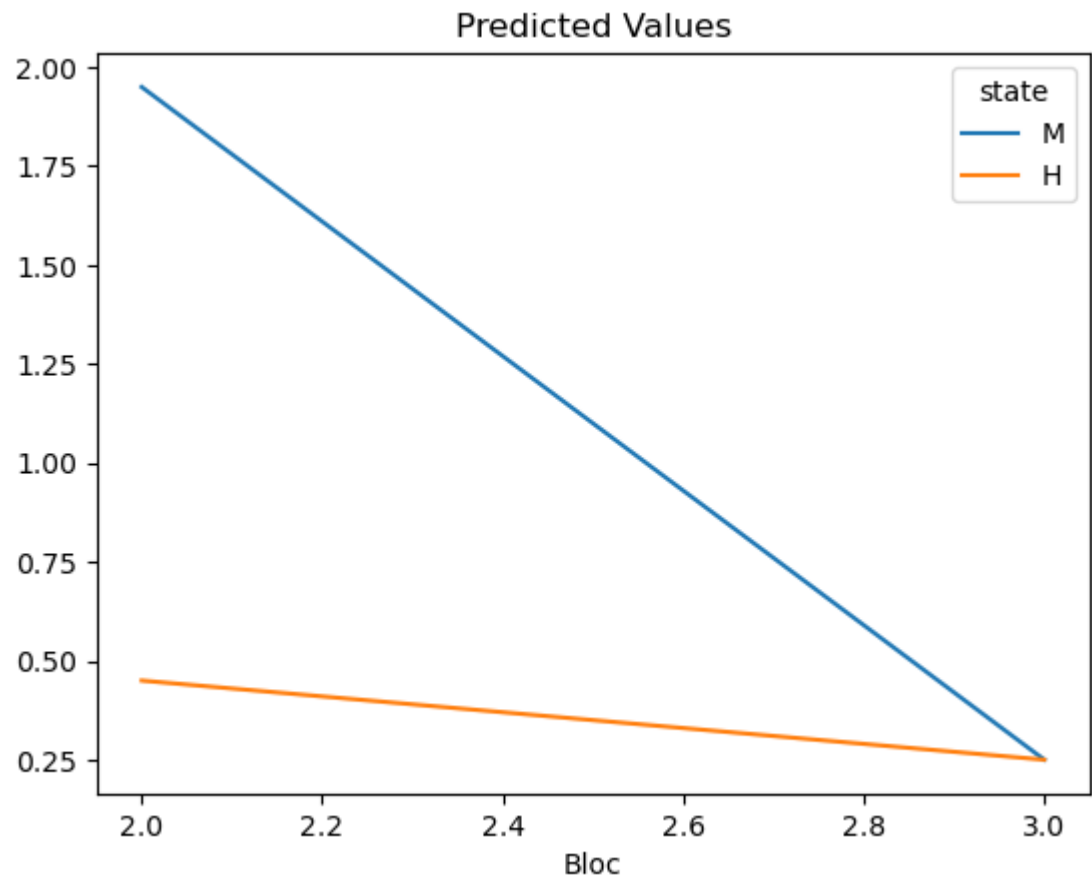
=====
=====
Omnibus:                21.547    Durbin-Watson:
1.795
Prob(Omnibus):          0.000    Jarque-Bera (JB):          4
1.090
Skew:                   -0.968    Prob(JB):                1.20
e-09
Kurtosis:               5.929    Cond. No.
39.1
=====
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

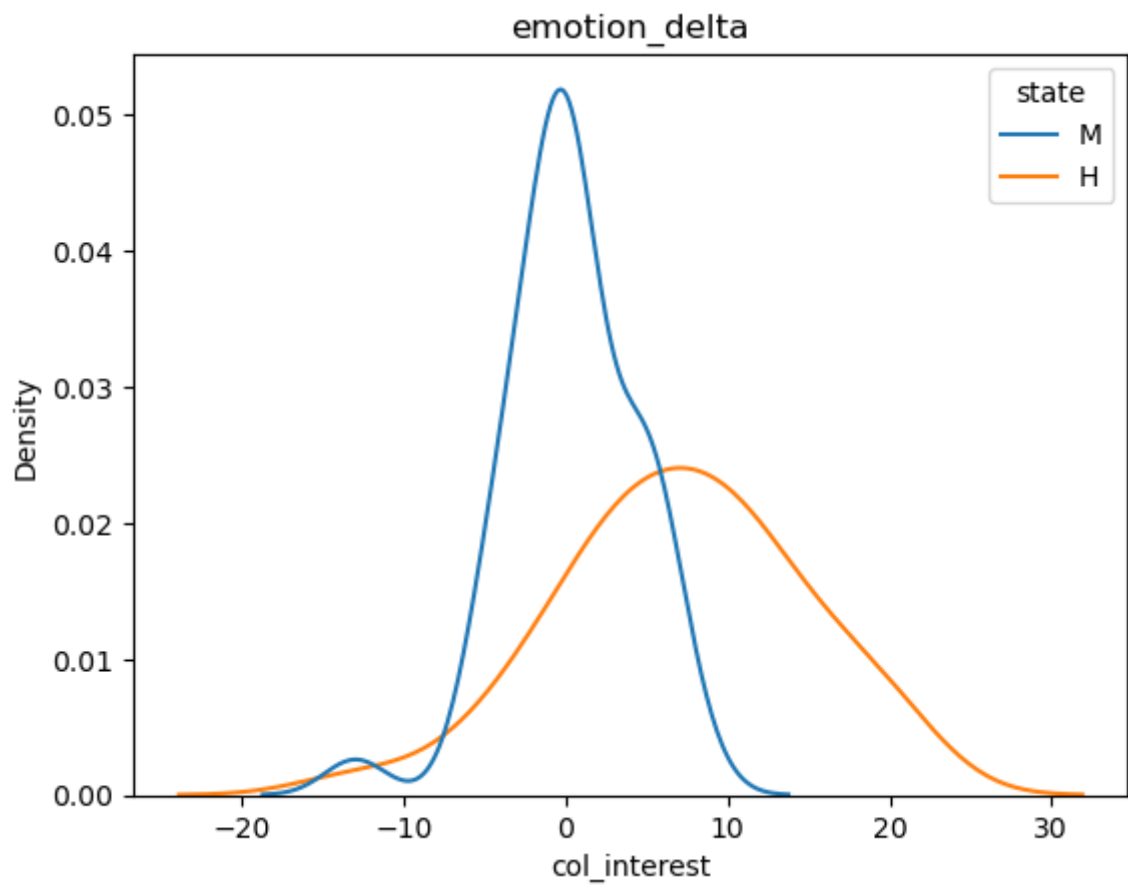
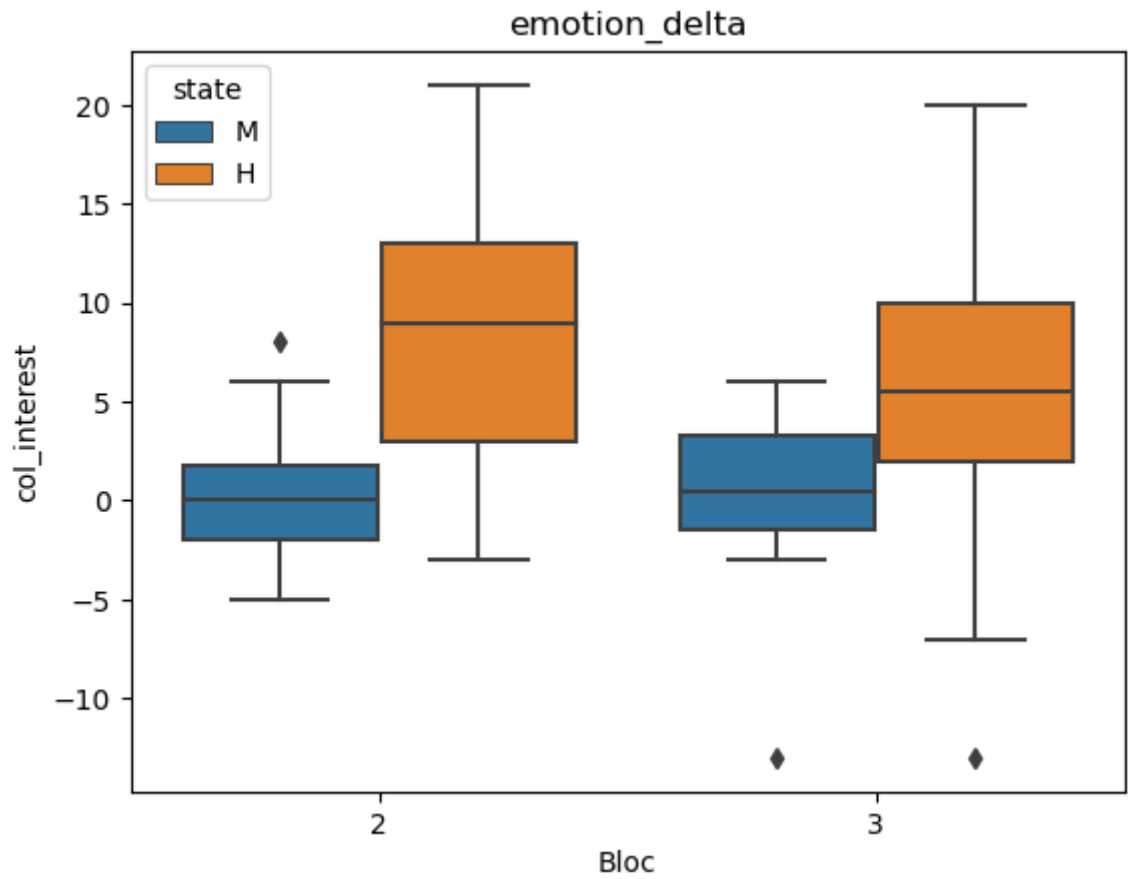
	sum_sq	df	F	PR(>F)
state	11.25	1.0	0.872983	0.353090
Bloc	18.05	1.0	1.400653	0.240303
state:Bloc	11.25	1.0	0.872983	0.353090
Residual	979.40	76.0	NaN	NaN



Multiple Comparison of Means – Tukey HSD, FWER=0.05

group1	group2	meandiff	p-adj	lower	upper	reject
H	M	0.75	0.3539	-0.8509	2.3509	False

FDR corrected p-values: [0.35385244]



OLS Regression Results

```

=====
=====
Dep. Variable:          col_interest    R-squared:
0.263
Model:                  OLS    Adj. R-squared:
0.234
Method:                Least Squares    F-statistic:
9.026
Date:                  Tue, 06 Jun 2023    Prob (F-statistic):          3.48
e-05
Time:                  15:32:33    Log-Likelihood:          -25
5.97
No. Observations:      80    AIC:          5
19.9
Df Residuals:          76    BIC:          5
29.5
Df Model:              3
Covariance Type:      nonrobust
=====
=====

```

```

=====
=====
              coef      std err          t      P>|t|      [0.025
0.975]
-----
Intercept      13.5000      4.909      2.750      0.007      3.724
23.276
state[T.M]     -13.6000      6.942     -1.959      0.054     -27.426
0.226
Bloc           -2.5500      1.925     -1.324      0.189      -6.385
1.285
state[T.M]:Bloc  2.7000      2.723      0.992      0.325      -2.723
8.123
=====
=====

```

```

=====
=====
Omnibus:          3.707    Durbin-Watson:
1.764
Prob(Omnibus):    0.157    Jarque-Bera (JB):
3.241
Skew:             -0.276    Prob(JB):
0.198
Kurtosis:         3.818    Cond. No.
39.1
=====
=====

```

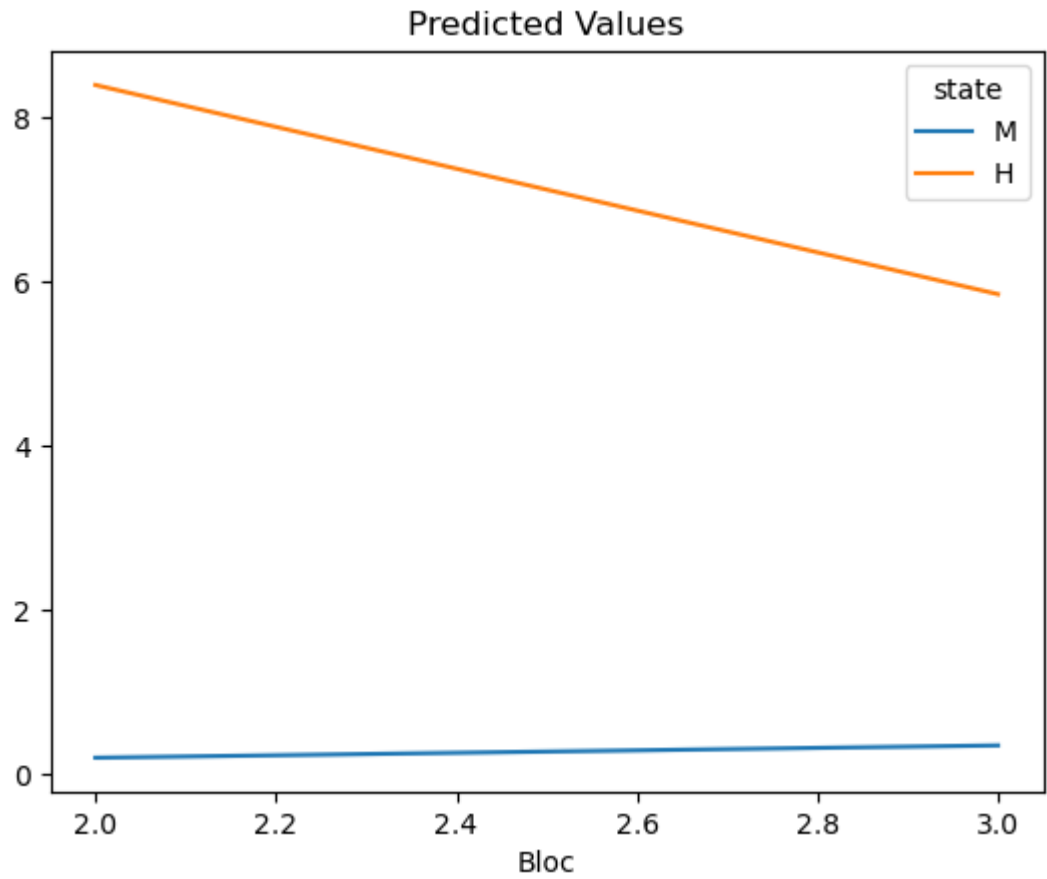
Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```

              sum_sq    df          F      PR(>F)
state      938.45    1.0  25.317596  0.000003
Bloc       28.80    1.0   0.776969  0.380849
state:Bloc  36.45    1.0   0.983352  0.324519
Residual  2817.10  76.0         NaN         NaN

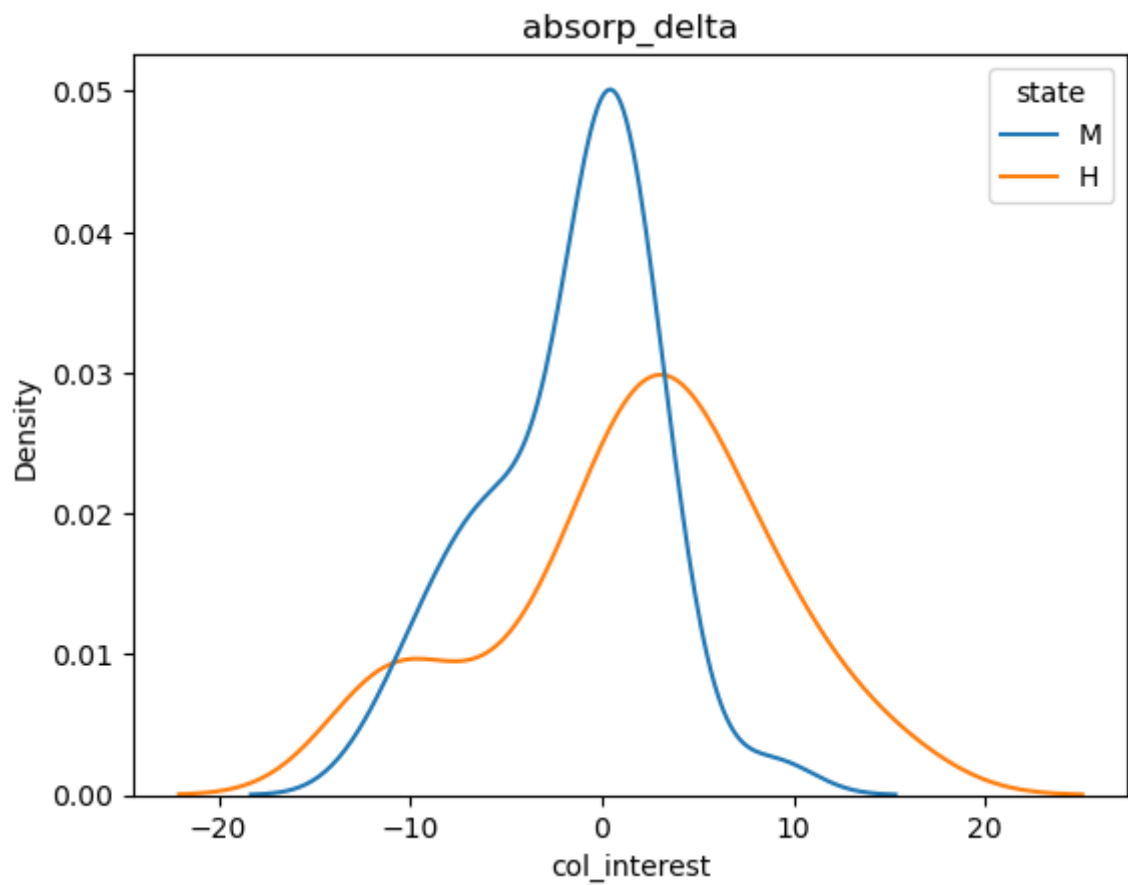
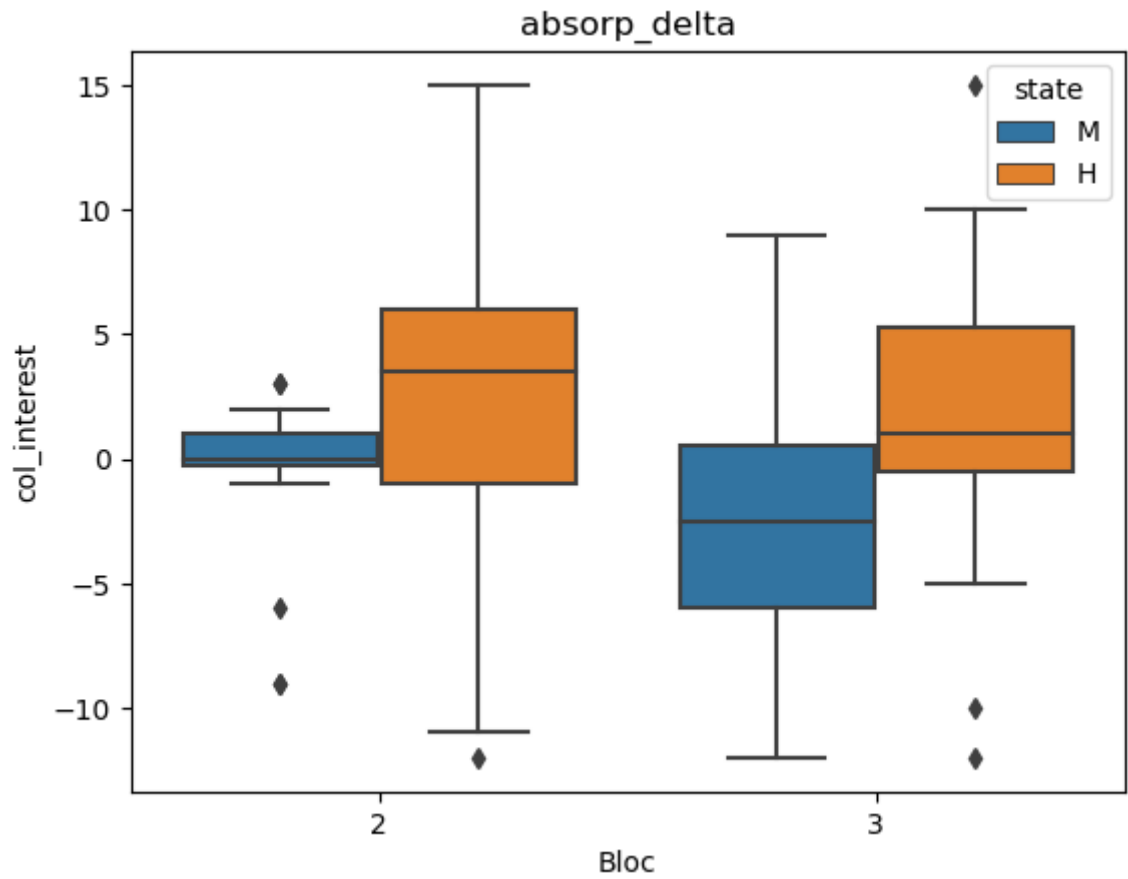
```



Multiple Comparison of Means – Tukey HSD, FWER=0.05

group1	group2	meandiff	p-adj	lower	upper	reject
H	M	-6.85	0.0	-9.5561	-4.1439	True

FDR corrected p-values: [2.95210492e-06]



OLS Regression Results

```

=====
=====
Dep. Variable:          col_interest    R-squared:
0.091
Model:                  OLS    Adj. R-squared:
0.055
Method:                 Least Squares    F-statistic:
2.547
Date:                  Tue, 06 Jun 2023    Prob (F-statistic):          0.
0622
Time:                  15:32:33    Log-Likelihood:          -25
3.30
No. Observations:      80    AIC:          5
14.6
Df Residuals:          76    BIC:          5
24.1
Df Model:              3
Covariance Type:       nonrobust
=====
=====

```

	coef	std err	t	P> t	[0.025
Intercept	2.2500	4.747	0.474	0.637	-7.204
state[T.M]	1.1500	6.713	0.171	0.864	-12.221
Bloc	-0.2000	1.862	-0.107	0.915	-3.908
state[T.M]:Bloc	-1.8000	2.633	-0.684	0.496	-7.044

```

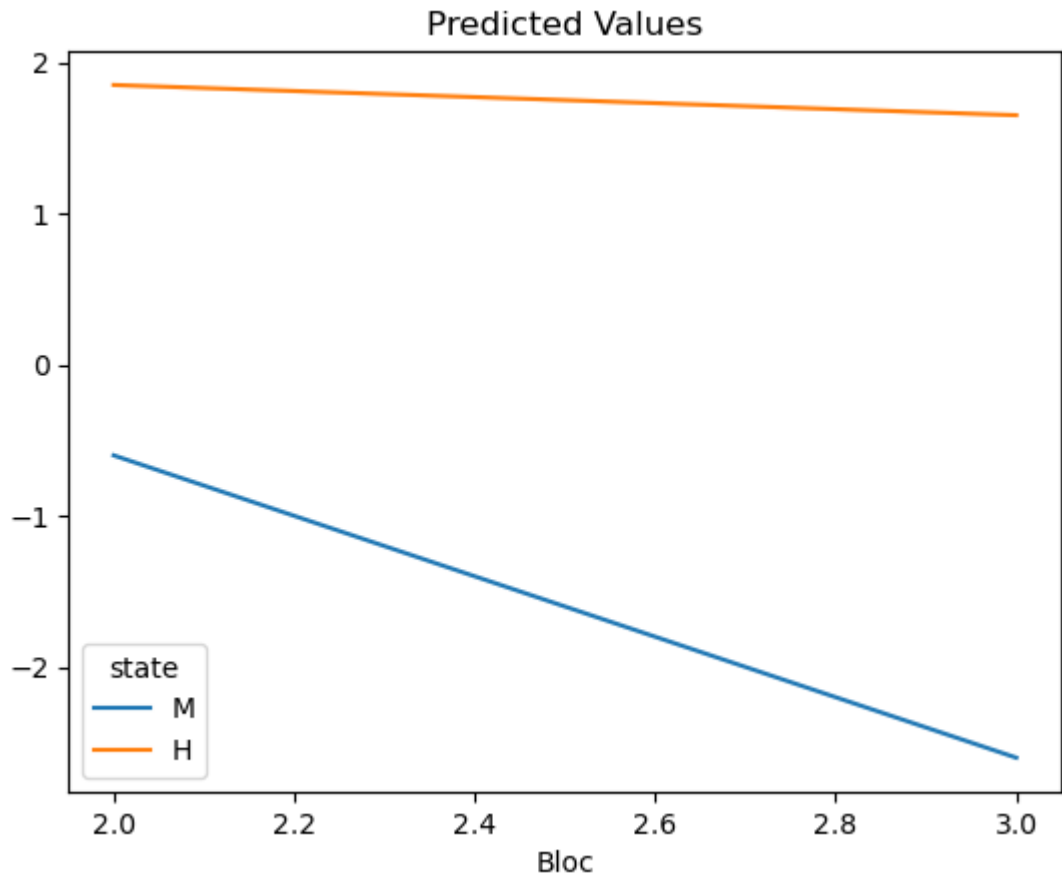
=====
=====
Omnibus:              3.031    Durbin-Watson:
1.951
Prob(Omnibus):        0.220    Jarque-Bera (JB):
2.293
Skew:                 -0.369    Prob(JB):
0.318
Kurtosis:             3.377    Cond. No.
39.1
=====
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

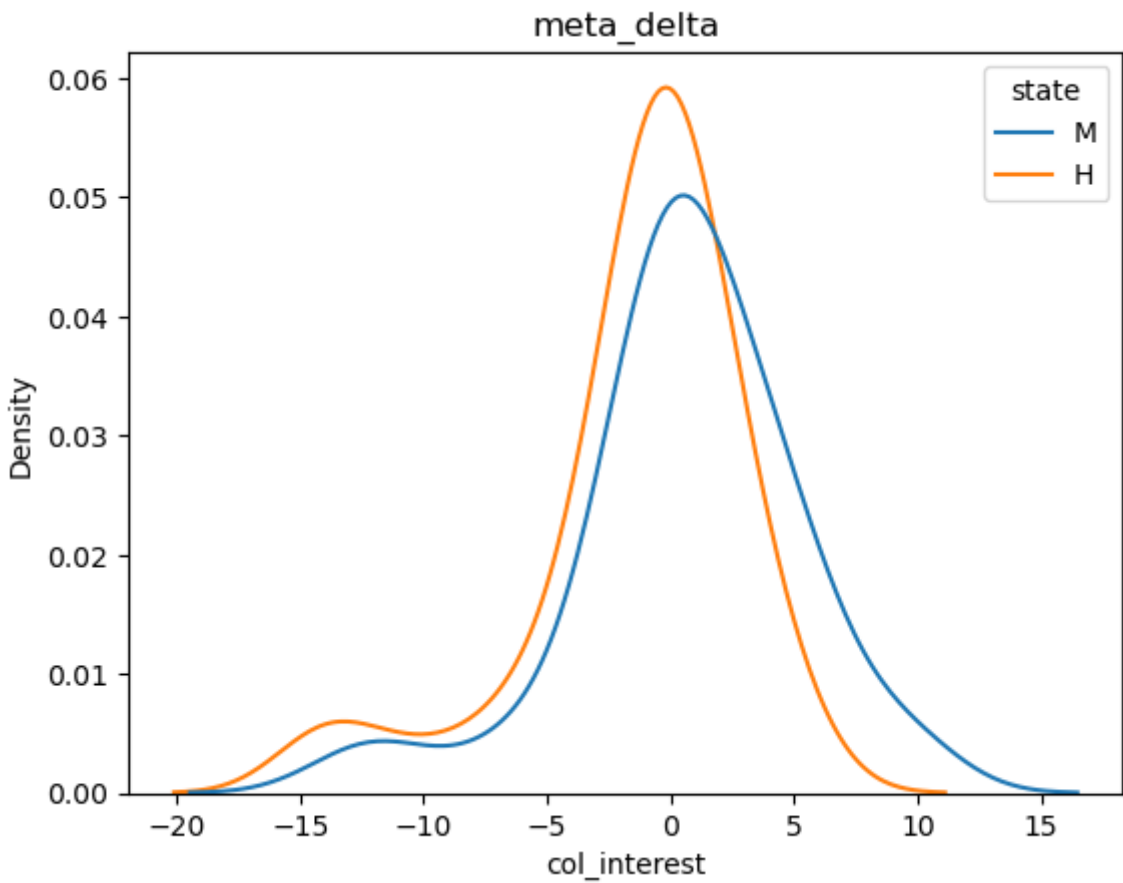
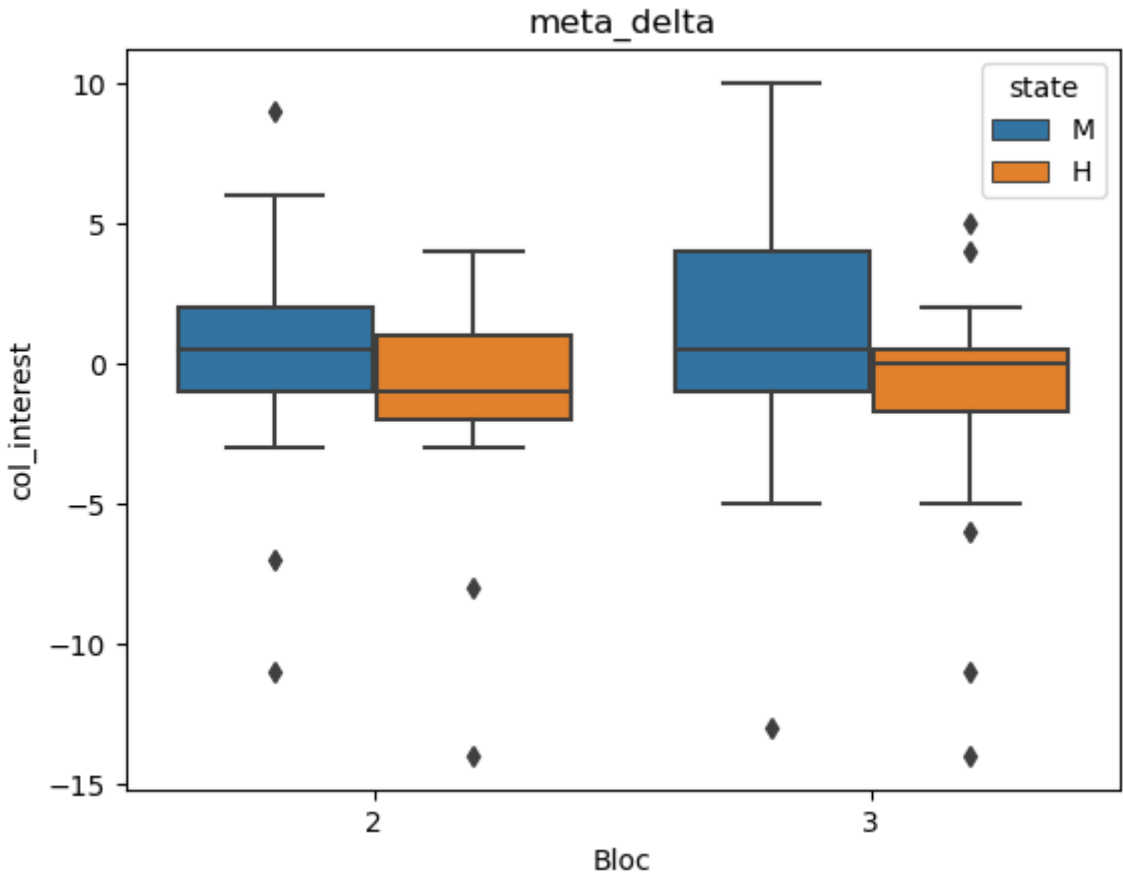
	sum_sq	df	F	PR(>F)
state	224.45	1.0	6.474437	0.012972
Bloc	24.20	1.0	0.698068	0.406053
state:Bloc	16.20	1.0	0.467302	0.496310
Residual	2634.70	76.0	NaN	NaN



Multiple Comparison of Means – Tukey HSD, FWER=0.05

group1	group2	meandiff	p-adj	lower	upper	reject
H	M	-3.35	0.0125	-5.957	-0.743	True

FDR corrected p-values: [0.01245808]



OLS Regression Results

```

=====
=====
Dep. Variable:          col_interest    R-squared:
0.050
Model:                  OLS             Adj. R-squared:
0.013
Method:                 Least Squares    F-statistic:
1.342
Date:                   Tue, 06 Jun 2023  Prob (F-statistic):
0.267
Time:                   15:32:34          Log-Likelihood:        -23
0.84
No. Observations:      80                AIC:                    4
69.7
Df Residuals:          76                BIC:                    4
79.2
Df Model:               3
Covariance Type:       nonrobust
=====
=====

```

```

=====
=====
                                coef      std err          t      P>|t|      [0.025
0.975]
-----
Intercept                    -0.8500      3.585        -0.237      0.813      -7.990
6.290
state[T.M]                   0.6000      5.070         0.118      0.906      -9.498
10.698
Bloc                        -0.2000      1.406        -0.142      0.887      -3.001
2.601
state[T.M]:Bloc              0.5500      1.989         0.277      0.783      -3.411
4.511
=====
=====

```

```

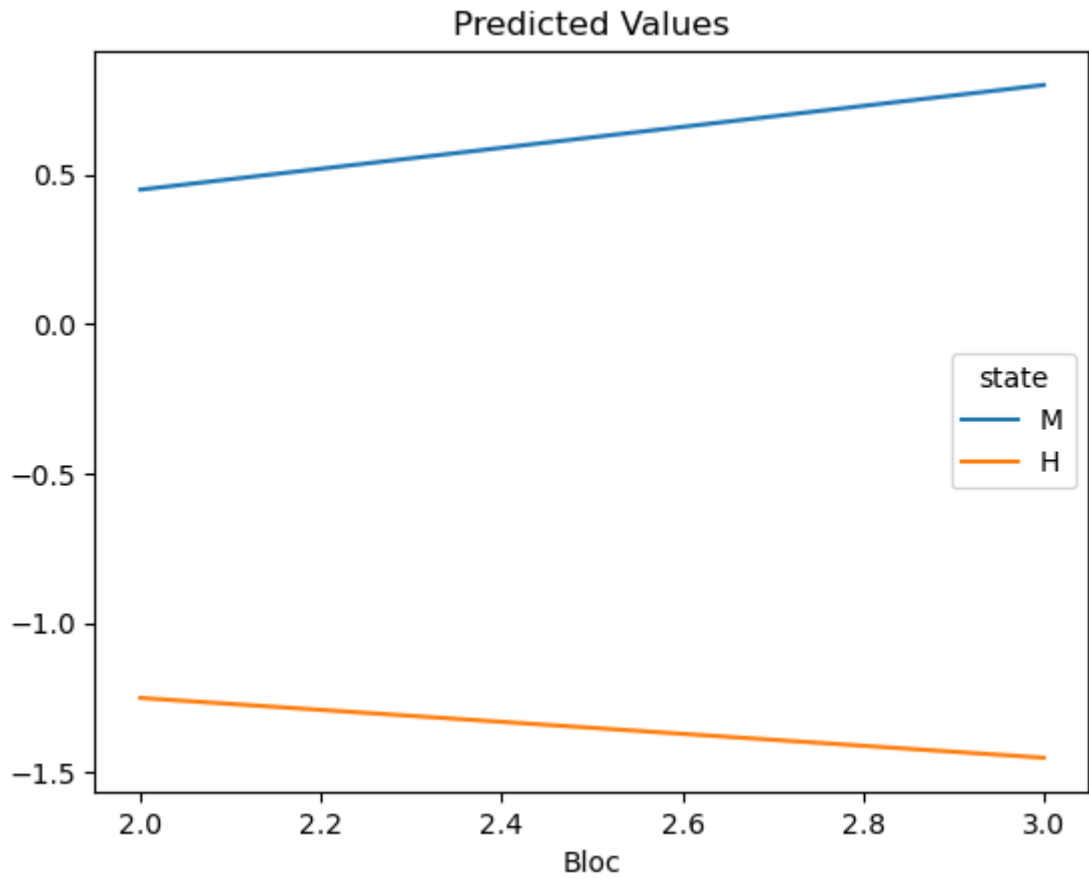
=====
=====
Omnibus:                  20.912    Durbin-Watson:
1.810
Prob(Omnibus):            0.000    Jarque-Bera (JB):        2
9.765
Skew:                     -1.116    Prob(JB):                3.44
e-07
Kurtosis:                 4.988    Cond. No.
39.1
=====
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

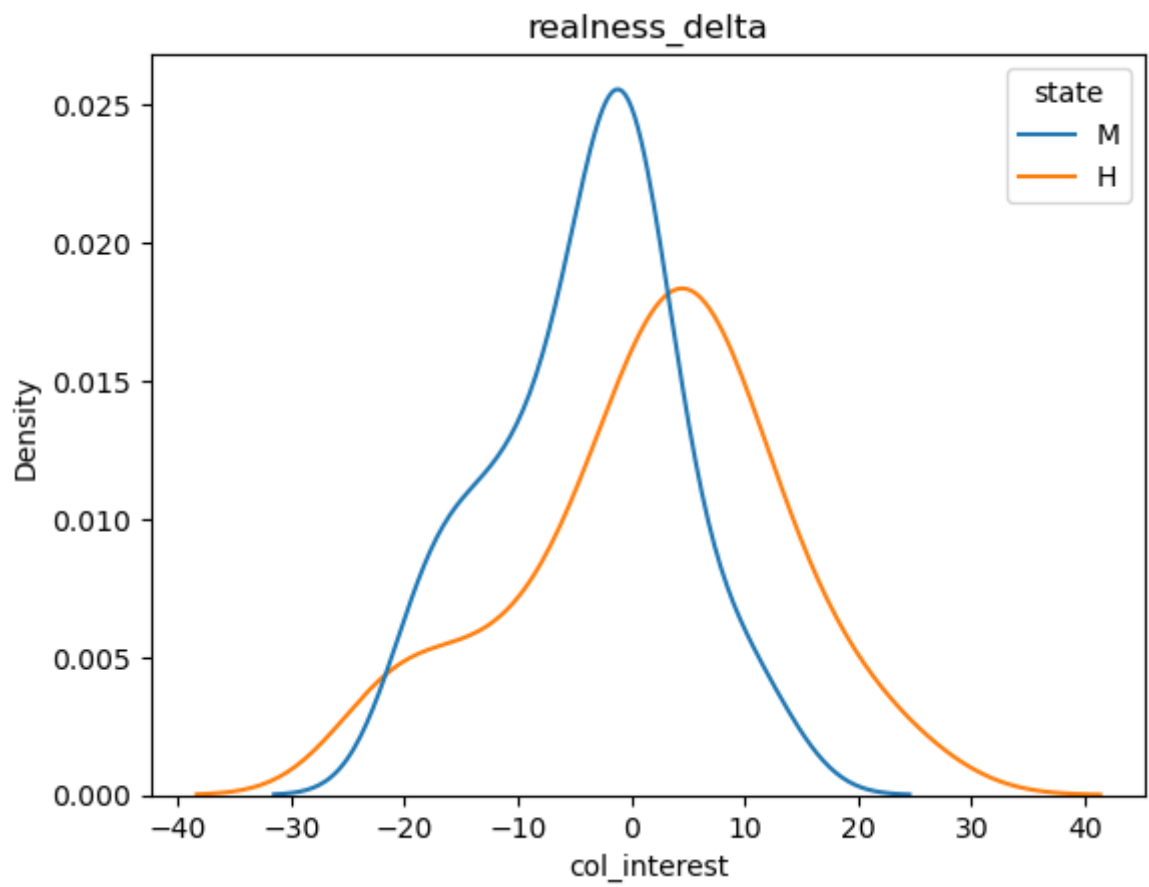
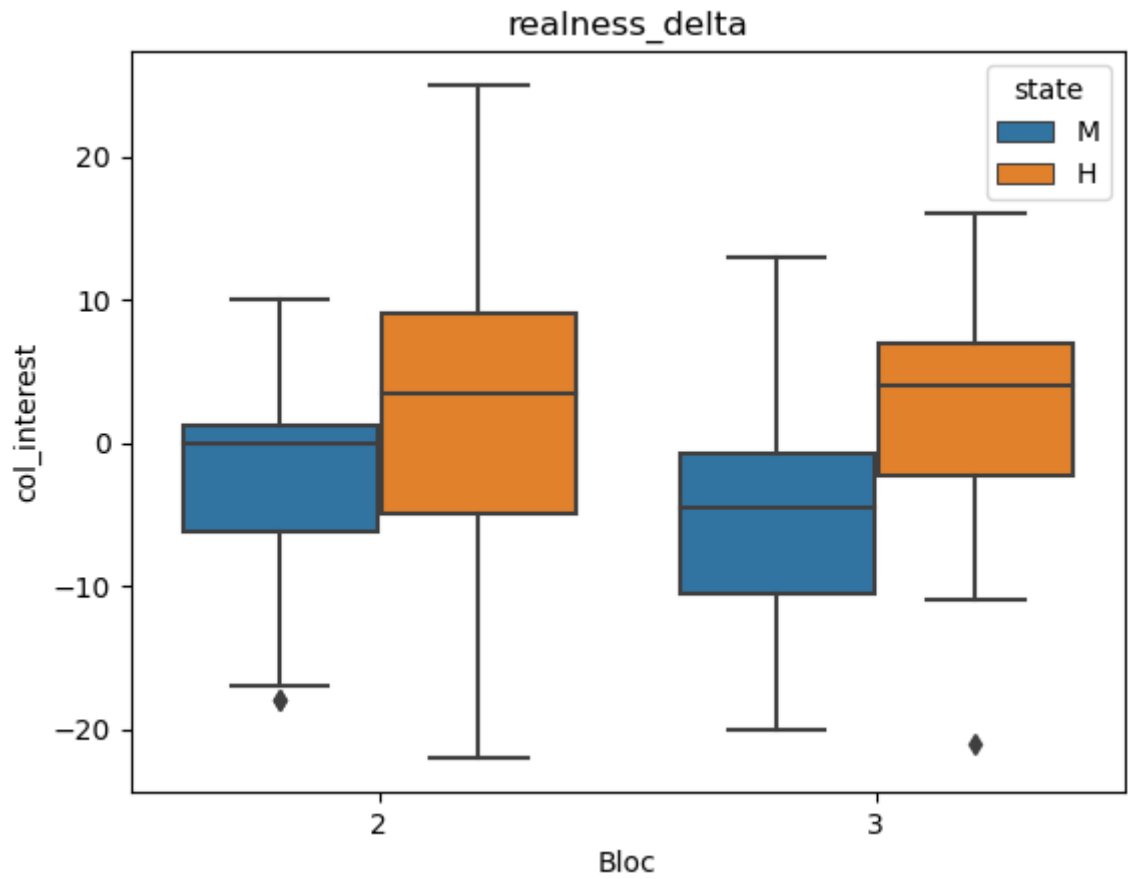
	sum_sq	df	F	PR(>F)
state	78.0125	1.0	3.945138	0.050613
Bloc	0.1125	1.0	0.005689	0.940073
state:Bloc	1.5125	1.0	0.076488	0.782865
Residual	1502.8500	76.0	NaN	NaN



Multiple Comparison of Means – Tukey HSD, FWER=0.05

group1	group2	meandiff	p-adj	lower	upper	reject
H	M	1.975	0.0478	0.0199	3.9301	True

FDR corrected p-values: [0.04776824]



OLS Regression Results

```

=====
=====
Dep. Variable:          col_interest    R-squared:
0.087
Model:                  OLS             Adj. R-squared:
0.051
Method:                 Least Squares    F-statistic:
2.404
Date:                  Tue, 06 Jun 2023    Prob (F-statistic):          0.
0740
Time:                  15:32:34           Log-Likelihood:          -29
5.12
No. Observations:      80                AIC:                    5
98.2
Df Residuals:          76                BIC:                    6
07.8
Df Model:              3
Covariance Type:       nonrobust
=====
=====

```

	coef	std err	t	P> t	[0.025
Intercept	1.8000	8.007	0.225	0.823	-14.147
state[T.M]	1.2500	11.324	0.110	0.912	-21.303
Bloc	-0.0500	3.141	-0.016	0.987	-6.305
state[T.M]:Bloc	-2.7500	4.442	-0.619	0.538	-11.596

```

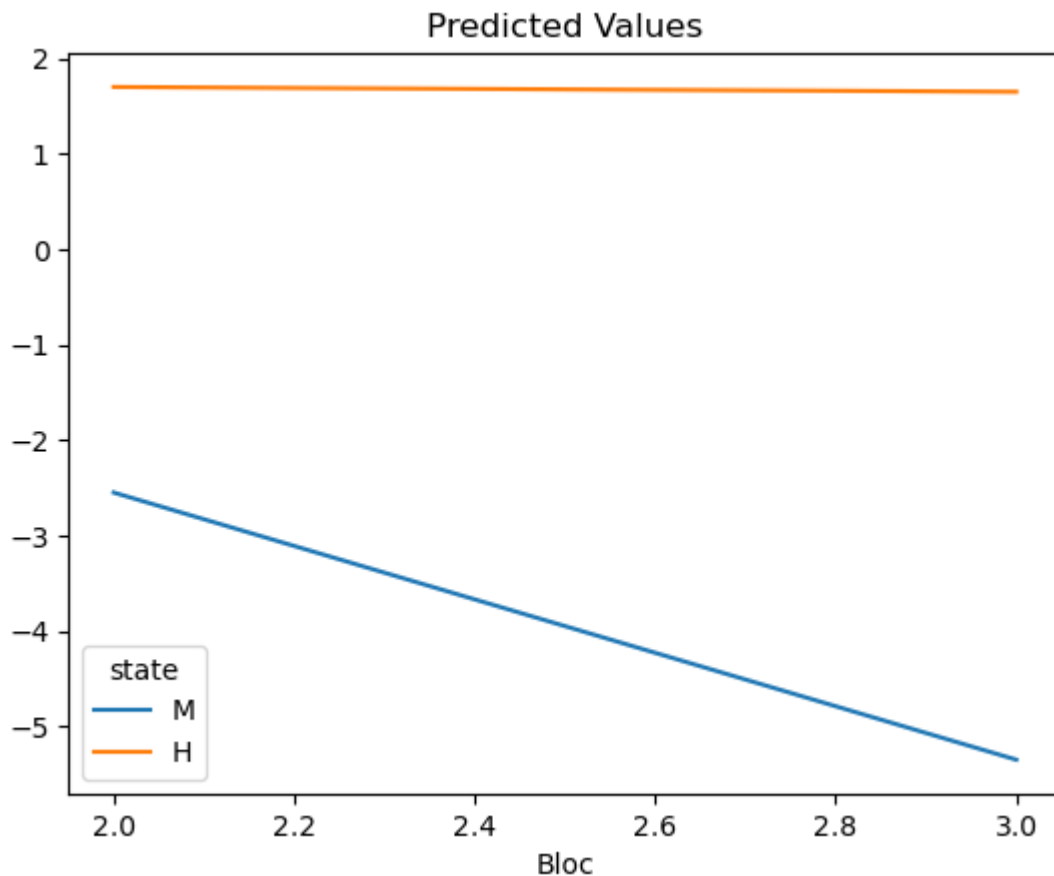
=====
=====
Omnibus:              2.485    Durbin-Watson:
2.029
Prob(Omnibus):        0.289    Jarque-Bera (JB):
1.965
Skew:                 -0.377    Prob(JB):
0.374
Kurtosis:             3.147    Cond. No.
39.1
=====
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

	sum_sq	df	F	PR(>F)
state	632.8125	1.0	6.415708	0.013373
Bloc	40.6125	1.0	0.411746	0.523015
state:Bloc	37.8125	1.0	0.383358	0.537663
Residual	7496.2500	76.0	NaN	NaN



Multiple Comparison of Means – Tukey HSD, FWER=0.05

group1	group2	meandiff	p-adj	lower	upper	reject
H	M	-5.625	0.0126	-10.0119	-1.2381	True

FDR corrected p-values: [0.01264066]

```
In [ ]: subset_A = data_delta[data_delta["Version"]=="A"] #participants with group A
subset_B = data_delta[data_delta["Version"]=="B"] #participants with group B

Bloc2 = data_delta[data_delta["Bloc"]==2]
Bloc3 = data_delta[data_delta["Bloc"]==3]

subset_A_Bloc2 = subset_A[subset_A["Bloc"]==2]
subset_B_Bloc2 = subset_B[subset_B["Bloc"]==2]

subset_A_Bloc3 = subset_A[subset_A["Bloc"]==3]
subset_B_Bloc3 = subset_B[subset_B["Bloc"]==3]

mean_difficulty_delta = pd.DataFrame({
    'Subset': ['Subset A', 'Subset B', 'Bloc 2', 'Bloc 3', 'Subset A Bloc 2', 'Subset A Bloc 3', 'Subset B Bloc 2', 'Subset B Bloc 3'],
    'Mean Difficulty Delta': [
        subset_A['difficulty_delta'].mean(),
        subset_B['difficulty_delta'].mean(),
        Bloc2['difficulty_delta'].mean(),
        Bloc3['difficulty_delta'].mean(),
        subset_A_Bloc2['difficulty_delta'].mean(),
        subset_B_Bloc2['difficulty_delta'].mean(),
        subset_A_Bloc3['difficulty_delta'].mean(),
        subset_B_Bloc3['difficulty_delta'].mean(),
    ]
})
```

```

        subset_B_Bloc3['difficulty_delta'].mean()
    ]
})

print(mean_difficulty_delta)

```

	Subset	Mean Difficulty Delta
0	Subset A	1.10
1	Subset B	0.35
2	Bloc 2	1.20
3	Bloc 3	0.25
4	Subset A Bloc 2	1.95
5	Subset B Bloc 2	0.45
6	Subset A Bloc 3	0.25
7	Subset B Bloc 3	0.25

Independent samples t-test (testing with difficulty)

```

In [ ]: import scipy.stats as stats

# performing a t-test to compare the mean differences between subset A and B
t_statistic, p_value = stats.ttest_ind(subset_A['difficulty_delta'], subset_B['difficulty_delta'])
print("t-statistic: ", t_statistic)
print("p-value: ", p_value)

# performing a t-test to compare the mean differences between Bloc 2 and Bloc 3
t_statistic, p_value = stats.ttest_ind(subset_A_Bloc2['difficulty_delta'], subset_A_Bloc3['difficulty_delta'])
print("t-statistic: ", t_statistic)
print("p-value: ", p_value)

# performing a t-test to compare the mean differences between Bloc 2 and Bloc 3
t_statistic, p_value = stats.ttest_ind(subset_A_Bloc2['difficulty_delta'], subset_B_Bloc3['difficulty_delta'])
print("t-statistic: ", t_statistic)
print("p-value: ", p_value)

t-statistic: 0.9327012357247281
p-value: 0.3538524442141706
t-statistic: 1.3431969686702177
p-value: 0.18717572333559673
t-statistic: 0.0
p-value: 1.0

```

Paired t-test (testing with difficulty)

```

In [ ]: # performing a paired t-test to compare the mean differences between Bloc 2 and Bloc 3
t_statistic, p_value = stats.ttest_rel(Bloc2['difficulty_delta'], Bloc3['difficulty_delta'])
print("t-statistic: ", t_statistic)
print("p-value: ", p_value)

# performing a paired t-test to compare the mean differences between Bloc 2 and Bloc 3
t_statistic, p_value = stats.ttest_rel(subset_A_Bloc2['difficulty_delta'], subset_A_Bloc3['difficulty_delta'])
print("t-statistic: ", t_statistic)
print("p-value: ", p_value)

# performing a paired t-test to compare the mean differences between Bloc 2 and Bloc 3
t_statistic, p_value = stats.ttest_rel(subset_A_Bloc2['difficulty_delta'], subset_B_Bloc3['difficulty_delta'])
print("t-statistic: ", t_statistic)
print("p-value: ", p_value)

```

```
print("t-statistic: ", t_statistic)  
print("p-value: ", p_value)
```

```
t-statistic: 1.1877974624930803  
p-value: 0.24209698499795473  
t-statistic: 1.27251182556806  
p-value: 0.21854404005791794  
t-statistic: -0.22615041624361465  
p-value: 0.8234994802702167
```